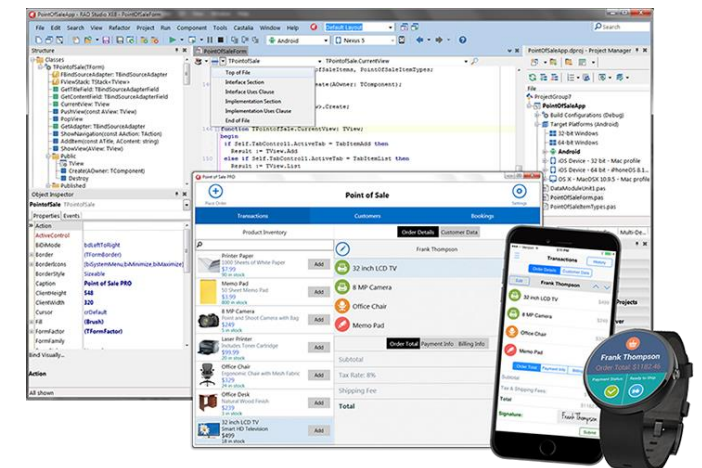
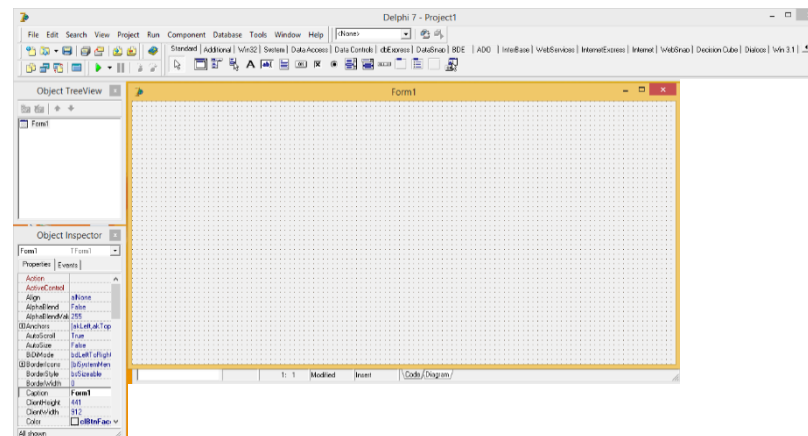
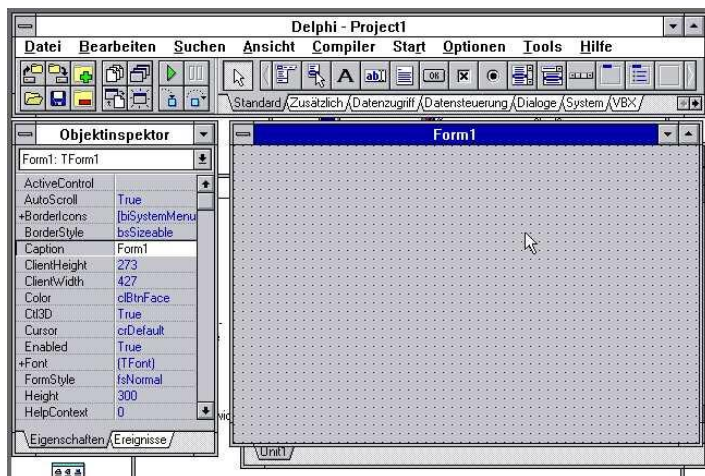


INTRODUÇÃO DELPHI

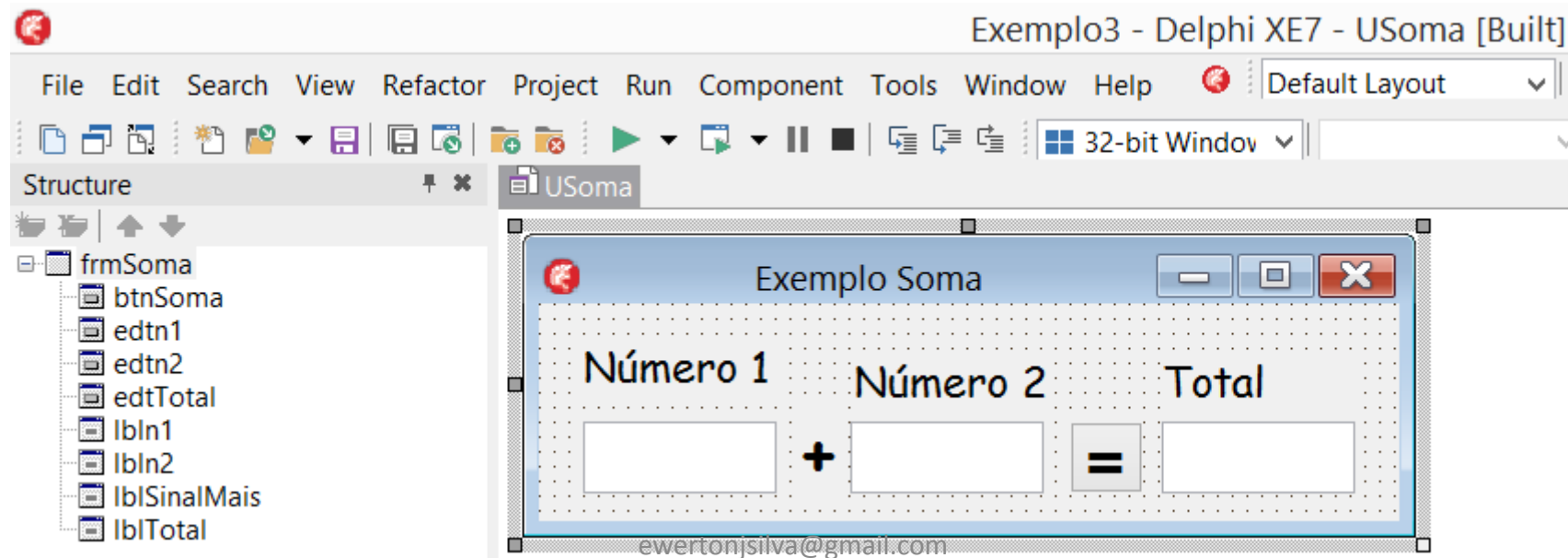
Ewerton J. Silva



Quando concatenar não funciona – Ex 3

Vimos que utilizando o sinal de + podemos combinar palavras inseridas em outro edit ou via código utilizando “

Crie um programa no Delphi como o abaixo, mantendo os mesmos nomes utilizados no exemplo para os componentes, form, unit e Project.



Quando concatenar não funciona

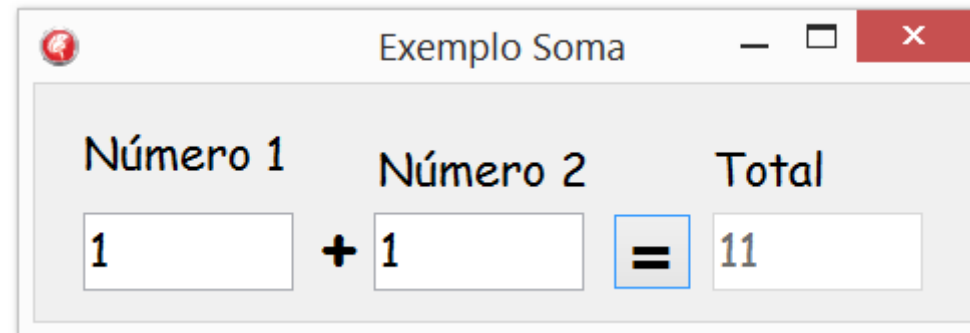
No componente btnSoma insira o código abaixo:

```
procedure TfrmSoma.btnSomaClick(Sender: TObject);  
begin  
    edtTotal.Text := edtn1.Text + edtn2.Text;  
end;
```

E teste o programa...

Quando concatenar não funciona

É fácil perceber que não existe nenhuma soma, o que ocorreu, foi que juntamos os dois valores digitados nos edits, pois todo valor digitado em um edit, seja ele letra ou número, é interpretado pelo Delphi como uma String. Uma String é um tipo de dado que é utilizado pelo Delphi para valores em formato de caracteres (Texto).



The screenshot shows a Delphi application window titled "Exemplo Soma". Inside the window, there are three text input fields labeled "Número 1", "Número 2", and "Total". The "Número 1" field contains the value "1", and the "Número 2" field also contains the value "1". Between these two fields is a plus sign "+". To the right of the "Número 2" field is an equals sign "=", and to its right is the "Total" field, which contains the value "11". This illustrates the result of concatenating the two strings "1" and "1" instead of performing a numerical addition.

Porque existem diferentes tipos de dados

Os tipos de dados são utilizados para definir quais são os tipos de valores que estão sendo manipulados, não podemos misturar texto com números, sendo assim não é possível somar o número 2 (do tipo inteiro) com o número 2 (do tipo String). Então lembre-se de que só é possível tratar dados se eles forem do mesmo tipo.

2 +   = %\$#erro*&@

FUNÇÕES DE CONVERSÃO E MANIPULAÇÃO

Então é impossível realizar cálculos no Delphi? Não, porém quando vamos realizar uma operação matemática por exemplo, temos que primeiro converter os valores que são do tipo String, para valores do tipo numérico, existem diversos tipos que podemos utilizar para armazenar valores numéricos, porém neste exemplo iremos trabalhar com o tipo inteiro. Para transformar uma String em Inteiro é necessário utilizar um recurso do Delphi que converte texto em número.

FUNÇÕES DE CONVERSÃO E MANIPULAÇÃO

Então já sabemos que os objetos do Delphi para entrada e/ou exibição de dados utilizam propriedades do tipo String, as propriedades Text e Caption são bons exemplos.

O problema ocorre quando tentamos realizar cálculos matemáticos com os dados que devem ser manipulados por estas propriedades.

Desta maneira precisamos de funções para converter dados String em tipos Inteiros ou Reais ou Datas, por exemplo.

Veja a seguir quais são as funções:

FUNÇÕES DE CONVERSÃO E MANIPULAÇÃO

Função	Objetivo
StrToInt()	Converte um dado String em tipo Inteiro
IntToStr()	Converte um dado Inteiro em tipo String
StrToFloat()	Converte um dado String em tipo Ponto Flutuante
FloatToStr()	Converte um dado Ponto Flutuante em tipo String
DateToStr()	Converte um dado TDateTime em String
DateTimeToStr()	Converte um dado TDateTime em String
StrToDate()	Converte um dado String em tipo TDateTime
StrToDateTime()	Converte um dado String em tipo TDateTime
FormatFloat()	Permite formatar um tipo ponto flutuante retornando uma String. Ex: edtTotal.Text := FormatFloat('###,###.00',soma); Sendo soma uma variável real.

O tipo TDateTime é internamente manipulado como tipo Ponto Flutuante.

FUNÇÕES DE CONVERSÃO E MANIPULAÇÃO

Agora iremos fazer com que o nosso programa some os valores digitados nos Edits ao invés de concatena-los. Insira o código abaixo no btnSoma.

```
procedure TfrmSoma.btnSomaClick(Sender: TObject);  
begin  
    edtTotal.Text := IntToStr(StrToInt(edtn1.Text) + StrToInt(edtn2.Text));  
end;
```

FUNÇÕES DE CONVERSÃO E MANIPULAÇÃO

O resultado é convertido para String

Sendo convertidos, os números serão somados

```
edtTotal.Text := IntToStr(StrToInt(edtn1.Text) + StrToInt(edtn2.Text));
```

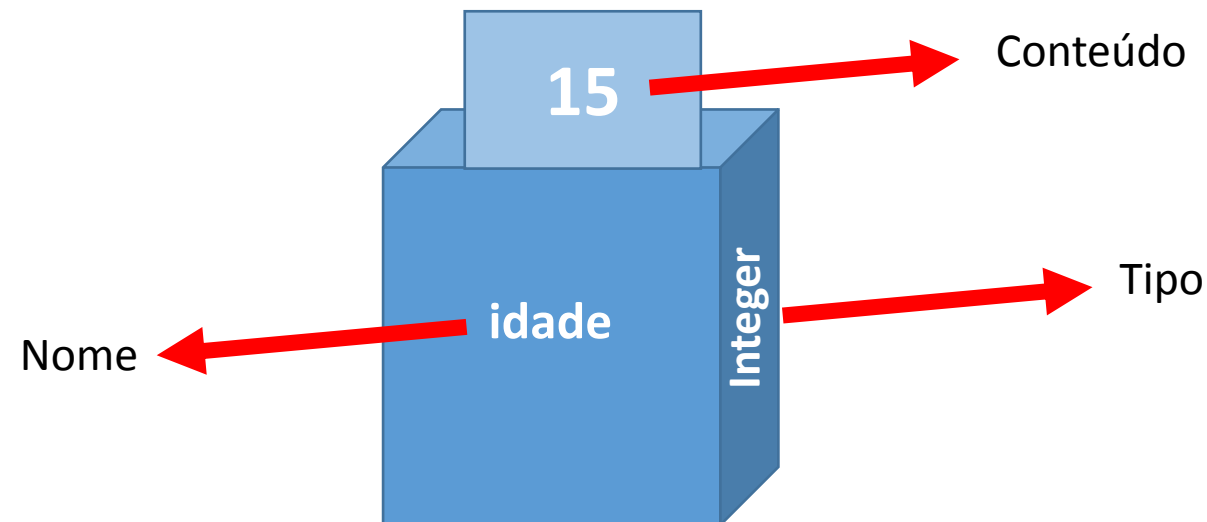
Converte o valor digitado no Edit no tipo inteiro

The diagram illustrates the execution of the code line `edtTotal.Text := IntToStr(StrToInt(edtn1.Text) + StrToInt(edtn2.Text));`. It uses red annotations to explain the steps: 1. A bracket under `edtTotal.Text` indicates it receives the final result. 2. An arrow points from `StrToInt(edtn1.Text)` to the text 'Converte o valor digitado no Edit no tipo inteiro'. 3. Another arrow points from `StrToInt(edtn2.Text)` to the text 'Sendo convertidos, os números serão somados'. 4. A third arrow points from `IntToStr` to the text 'O resultado é convertido para String'.

O componente `edtTotal` recebe o valor convertido em sua propriedade `text`

Variáveis

No exemplo anterior vimos que é possível realizar operações usando apenas os conteúdos dos Edits, porém é algo simples de se utilizar quando a conta é pequena, em cálculos mais complexos tendem a aumentar o tamanho do código e isto pode atrapalhar. Para tornar o código mais simples de entender, de manipular e eficiente utilizamos variáveis.



Variáveis

Uma variável corresponde a uma “região de memória (do computador) previamente identificada cuja finalidade é armazenar os dados ou informações de um programa por um determinado espaço de tempo”. A memória do computador se organiza tal qual um armário com várias divisões. Sendo cada divisão identificada por um endereço diferente em uma linguagem que o computador entende.

O computador armazena os dados nessas divisões, sendo que em cada divisão só é possível armazenar um dado e toda vez que o computador armazenar um dado em uma dessas divisões, o dado que antes estava armazenado é eliminado. O conteúdo pode ser alterado, mas somente um dado por vez pode ser armazenado naquela divisão.

O computador identifica cada divisão por intermédio de um endereço no formato hexadecimal, e as linguagens de programação permitem nomear cada endereço ou posição de memória, facilitando a referência a um endereço de memória. Uma variável é composta por dois elementos básicos: o **conteúdo**, o valor da variável e **identificador**, um nome dado à variável para possibilitar sua utilização.

Variáveis

No Delphi, toda variável tem que ser declarada antes de ser utilizada,. Essas declarações devem ser feitas após a palavra reservada “var”, onde são indicados o nome e o tipo da variável.

Os nomes de variáveis não podem ter acentos, espaços ou caracteres especiais como &,\$ ou % e o primeiro caractere de um nome de variável tem que ser uma letra ou um sublinhado.

Variáveis

Existem dois tipos de variáveis quanto a sua “vida útil”: variável global e variável local.

- Global: quando declarada em uma unit terá vida útil enquanto essa unit existir, ou seja, enquanto o programa estiver rodando.

Quando elas são feitas diretamente na seção *interfaces* de uma unit (ou seja, fora dos procedimentos e funções). Pode-se ter variáveis *públicas* e *privadas*.

- Variáveis Públicas: Podem ser utilizadas em outras Units
- Variáveis Privadas: Só poderão ser utilizadas na mesma Unit onde foram criadas.
- Local: tem vida somente dentro daquele bloco de código em que foi declarada. Após o compilador passar pelo código, ela não tem mais valor a ser verificado.

Evite o uso de variáveis globais, isso pode ter efeito de performance em seu sistema.

Variáveis GLOBAIS

- São declaradas na Interface da Unit e podem ser acessadas por qualquer Unit.

```
unit UVariaveis;  
  
interface  
  
uses  
    Winapi.Windows, Winapi.Messages, System.SysUtils, System.Variants,  
    System.Classes, Vcl.Graphics,  
    Vcl.Controls, Vcl.Forms, Vcl.Dialogs, Vcl.StdCtrls;  
  
type  
    TfrmVariaveis = class(TForm)  
        btnCodigo: TButton;  
    private  
        { Private declarations }  
    public  
        { Public declarations }  
    end;  
  
var  
    frmVariaveis: TfrmVariaveis;  
  
implementation  
  
{$R *.dfm}  
  
end.
```



Variáveis LOCAIS

- Local: tem vida somente dentro daquele bloco de código em que foi declarada.

```
unit UVariaveis;

interface

uses
  Winapi.Windows, Winapi.Messages, System.SysUtils, System.Variants,
  System.Classes, Vcl.Graphics,
  Vcl.Controls, Vcl.Forms, Vcl.Dialogs, Vcl.StdCtrls;

type
  TfrmVariaveis = class(TForm)
    btnCodigo: TButton;
    procedure btnCodigoClick(Sender: TObject);
  private
    { Private declarations }
  public
    { Public declarations }
  end;
var
  frmVariaveis: TfrmVariaveis;

implementation

{$R *.dfm}

procedure TfrmVariaveis.btnCodigoClick(Sender: TObject);
begin
end;

end.
```

Variáveis LOCAIS – Ex 4

```
implementation

{$R *.dfm}

procedure TfrmVariaveis.btnCodigoClick(Sender: TObject);
var
    mensagem : String;
begin
    mensagem := 'Uma variável local só pode ser acessada no bloco ' +
                'de comando onde ela foi criada';
    ShowMessage(mensagem);
end;

end.
```

Variáveis LOCAIS – Ex 4

```
implementation
```

```
{ $R *.dfm }
```

```
procedure TfrmVariaveis.btnCodigoClick(Sender: TObject);
```

```
var
```

```
    mensagem : String;
```

```
begin
```

```
    mensagem := 'Uma variável local só pode ser acessada no bloco ' +  
                'de comando onde ela foi criada';
```

```
    ShowMessage(mensagem);
```

```
end;
```

```
procedure TfrmVariaveis.btnErroClick(Sender: TObject);
```

```
begin
```

```
    mensagem := 'A variável mensagem não foi criada neste bloco ' +  
                'e sim no bloco referente ao btnCodigo, sendo ' +  
                'assim ocorre um erro no programa, pois a variável ' +  
                'não existe!';
```

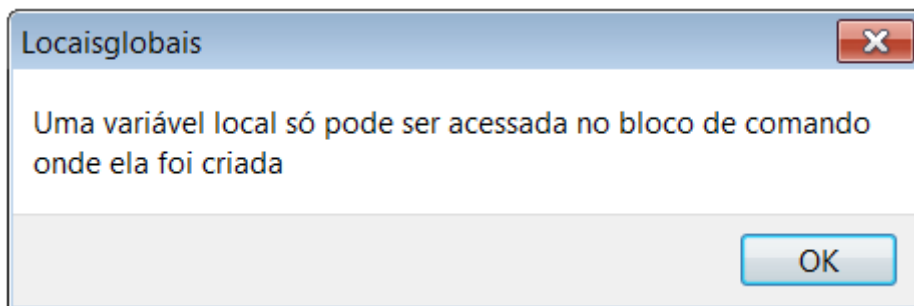
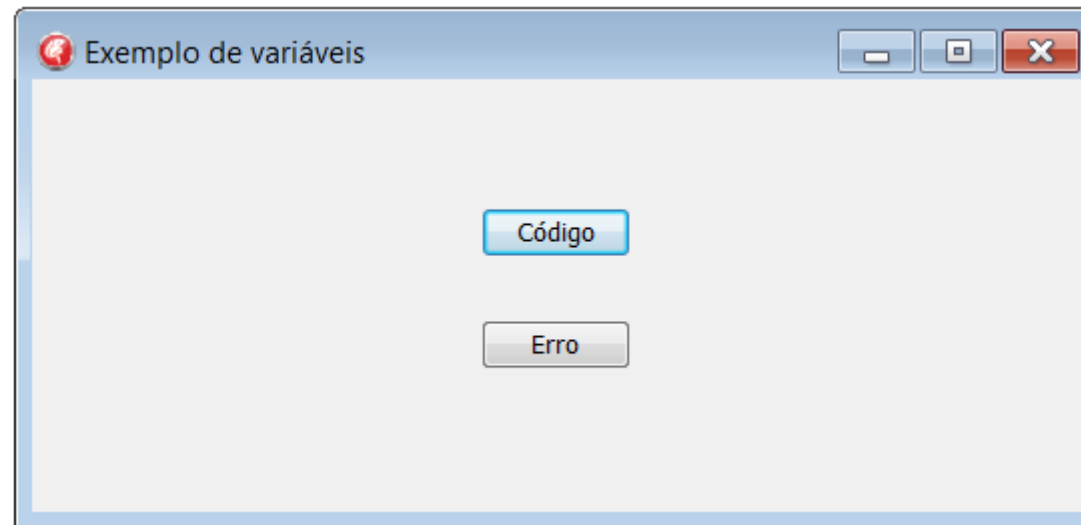
```
    ShowMessage(mensagem);
```

```
end;
```

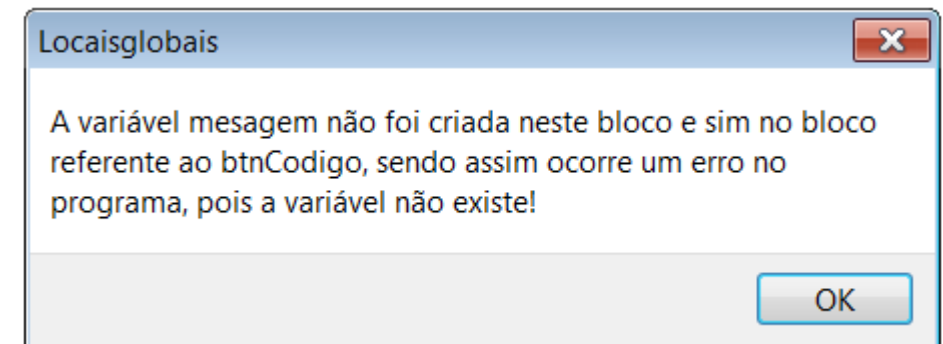
```
end.
```

Exercícios

24. Arrume o programa demonstrado no exemplo anterior.



ewertonjsilva@gmail.com



Constantes

Uma constante é um identificador com valor(es) fixo(s). Um bloco de declarações constante possui a seguinte expressão:

[Declaração Constante] [Identificador] (=) [constante] (;)

A lista abaixo apresenta um conjunto de funções que podem ser utilizadas para a declaração das constantes:

Ab	Length	Ord	SizeOf
Chr	Lo	Pred	Succ
Hi	Low	Ptr	Swap
High	Odd	Round	Trunc

Constantes

Alguns exemplos para a definição de Constantes:

```
const Min = 0;  
Max = 100;  
Centro = (Max - Min) div 2;  
Beta = Chr(225);  
NumLetras = Ord('Z') - Ord('A') + 1;  
MensOla = 'Instrução inválida';  
MensErro = ' Erro: ' + MensOla + ' . ';  
PosErr = 80 - Length(MensErro) div 2;  
Ln10 = 2.302585092994045684;  
Ln10R = 1 / Ln10;  
DigNumericos = ['0'..'9'];  
LetrasAlpha = ['A'..'Z', 'a'..'z'];  
AlphaNum = LetrasAlpha + DigNumericos;
```

Tipos de variáveis Inteiras



<i>Tipo</i>	<i>Faixa de Valores</i>	<i>Formato</i>
Integer	-2147483648.. 2147483647	32 bits
Cardinal	0..4294967295	32 bits, sem sinal
Shortint	-128..127	8 bits
Smallint	-32768..32767	16
Longint	-2147483648.. 2147483647	32
Int64	-2 ⁶³ ..2 ⁶³ -1	64
Byte	0..255	8 bits, sem sinal
Word	0..65535	16 bits, sem sinal
Longword	0..4294967295	32 bits, sem sinal

Tipos de números Reais

Tipos Reais



Tipo	Domínio	Alg. Sig.	Tam.
Real	2.9×10^{-39} à 1.7×10^{38}	11-12	6
Single	1.5×10^{-45} à 3.4×10^{38}	7-8	4
Double	5.0×10^{-324} à 1.7×10^{308}	15-16	8
Extended	3.4×10^{-4932} à 1.1×10^{4932}	19-20	10
Comp	$-2^{63}+1$ à $2^{63}-1$	19-20	8
Currency	-922337203685477.5808 à 922337203685477.5807	19-20	8

Tipos de variáveis booleanas

Tipos Booleanos



Tipo	Descrição
Boolean	1 byte, assume apenas valores TRUE ou FALSE
ByteBool	É um byte com características booleanas, 0(FALSE) e diferente de 0 (TRUE)
WordBool	É um Word com características booleanas, 0(FALSE) e diferente de 0 (TRUE)
LongBool	É um Longint com características booleanas, 0(FALSE) e diferente de 0 (TRUE)

Tipos de variáveis de caracteres

<i>Tipo</i>	<i>Valores</i>
Char	Permite armazenar um caractere ASCII.
ShortString	Permite armazenar uma cadeia de até 255 caracteres.
String	Permite armazenar uma cadeia 'ilimitada' de caracteres.



Formas de Atribuição

- O Delphi utiliza a mesma forma de atribuição do Pascal (:=).
- A única diferença é que a atribuição foi estendida aos objetos e aos novos tipos do Delphi. Ex.:

`X := 10 + Y;`

`Form1 := Form2;`

Operadores

- Operadores: O Delphi Possui uma estrutura de operadores muito parecida com a do pascal, apenas com a inclusão de alguns novos operadores.

Operadores Aritméticos

Operador	Operação	Tipos Usados	Tipos Resultantes
+	Adição	Inteiros Reais	Inteiro Real
-	Subtração	Inteiros Reais	Inteiro Real
*	Multiplicação	Inteiros Reais	Inteiro Real
/	Divisão	Inteiros Reais	Real Real
Div	Divisão inteira	Inteiros	Inteiro
Mod	Resto da divisão inteira	Inteiros	Inteiro

Operadores

Operadores Booleanos

Operador	Operação	Tipos Usados	Tipos Resultantes
Not	Negação	Booleanos	Booleano
And	E	Booleanos	Booleano
Or	OU	Booleanos	Booleano
Xor	OU coincidente	Booleanos	Booleano

Operadores Relacionais

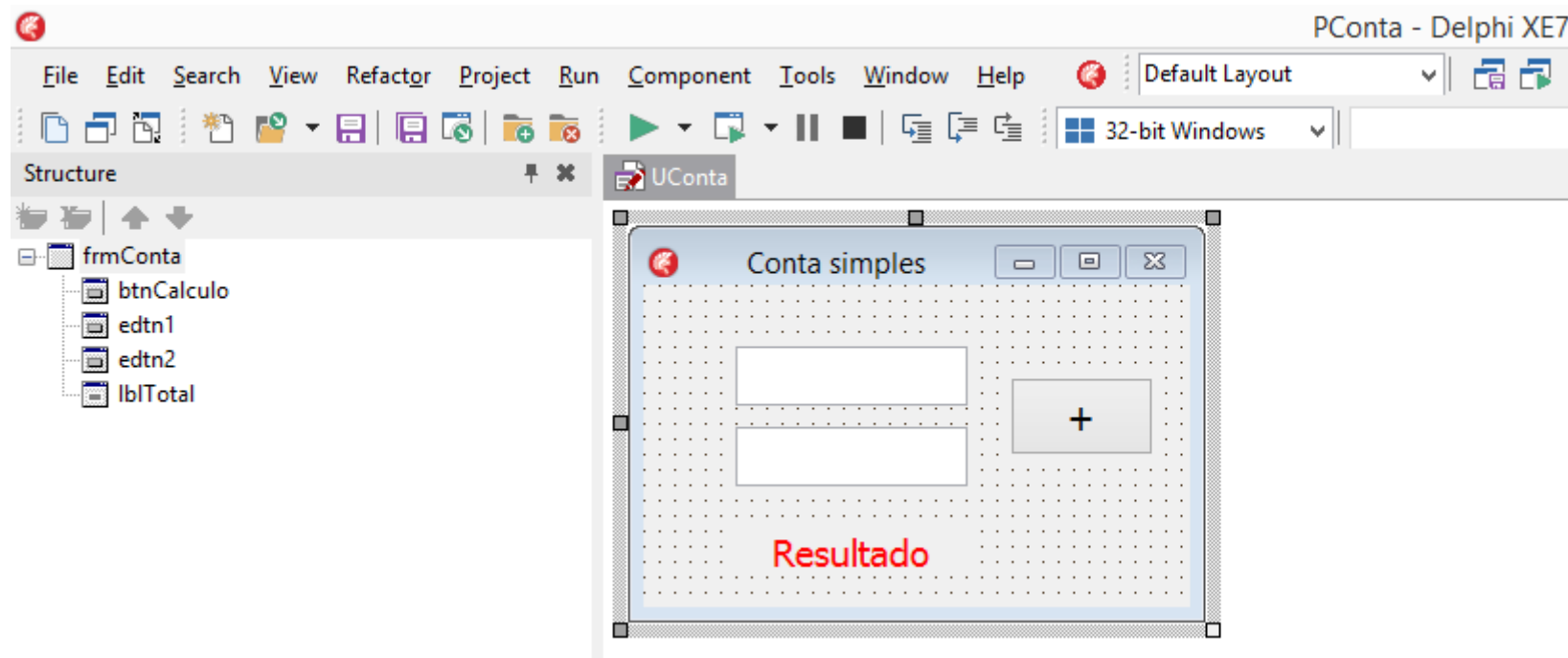
Operador	Operação	Tipos Usados	Tipos Resultantes
=	Igual	Tipos simples compatíveis, de classe, referencia de classe, ponteiros, conjunto, string ou string empacotado.	Booleano
Operador	Operação	Tipos Usados	Tipos Resultantes
<>	Diferente de	Tipos simples compatíveis, de classe, referencia de classe, ponteiros, conjunto, string ou string empacotado.	Booleano

Operadores Relacionais

<	Menor que	Tipos simples compatíveis, strings ou string empacotado ou Pchar	Booleano
>	Maior que	Tipos simples compatíveis, strings ou string empacotado ou Pchar	Booleano
<=	Menor ou igual	Tipos simples compatíveis, strings ou string empacotado ou Pchar	Booleano
>=	Maior ou igual	Tipos simples compatíveis, strings ou string empacotado ou Pchar	Booleano
<=	Subconjunto de	Tipos de conjuntos compatíveis	Booleano
>=	Superconjunto de	Tipos de conjuntos compatíveis	Booleano

Cálculos com variáveis – Ex 5

- Agora iremos realizar um cálculo simples utilizando variáveis, crie um novo projeto como o do exemplo abaixo.



Cálculos com variáveis – Ex 5

- Para realizar o cálculo é necessário que sejam criadas três variáveis, uma para armazenar o valor 1, outra para armazenar o valor 2 e a terceira para armazenar o resultado da conta.

```
procedure TfrmConta.btnCalculoClick(Sender: TObject);  
var  
→ n1, n2, total : Integer;  
begin  
    n1 := StrToInt(edtn1.Text);  
    n2 := StrToInt(edtn2.Text);  
  
    total := n1 + n2;  
end;
```

Cálculos com variáveis – Ex 5

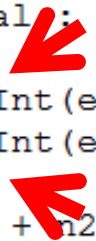
- Nosso primeiro cálculo envolve números do tipo inteiro, o tipo de dados deve ser declarado junto com as variáveis.

```
procedure TfrmConta.btnCalculoClick(Sender: TObject);  
var  
    n1, n2, total : Integer;   
begin  
    n1 := StrToInt(edtn1.Text);  
    n2 := StrToInt(edtn2.Text);  
  
    total := n1 + n2;  
end;
```

Cálculos com variáveis – Ex 5

- Todo valor digitado em um edit é considerado texto, porém a variável onde o valor será armazenado é do tipo Inteiro. Não é possível armazenar caracteres em uma variável do tipo Inteiro, mas o Delphi oferece o recurso de conversão de dados, para converter os caracteres em números vamos utilizar o código StrToInt, que converte os números digitados no Edit para valores do tipo Inteiro.

```
procedure TfrmConta.btnCalculoClick(Sender: TObject);  
var  
    n1, n2, total : Integer;  
begin  
    n1 := StrToInt(edtn1.Text);  
    n2 := StrToInt(edtn2.Text);  
  
    total := n1 + n2;  
end;
```



Cálculos com variáveis – Ex 5

- Nas duas linhas apontadas abaixo estamos armazenando nas variáveis n1 e n2 o valor convertido para Inteiro dos Edits

```
procedure TfrmConta.btnCalculoClick(Sender: TObject);  
var  
    n1, n2, total : Integer;  
begin  
    n1 := StrToInt(edtn1.Text);  
    n2 := StrToInt(edtn2.Text);  
  
    total := n1 + n2;  
end;
```



Cálculos com variáveis – Ex 5

- Para realizar um cálculo, lembre-se sempre de que é necessário apontar onde o resultado será armazenado. Em nosso exemplo iremos armazenar o resultado do cálculo na variável total.

```
procedure TfrmConta.btnCalculoClick(Sender: TObject);  
var  
    n1, n2, total : Integer;  
begin  
    n1 := StrToInt(edtn1.Text);  
    n2 := StrToInt(edtn2.Text);  
    total := n1 + n2;  
end;
```

Cálculos com variáveis – Ex 5

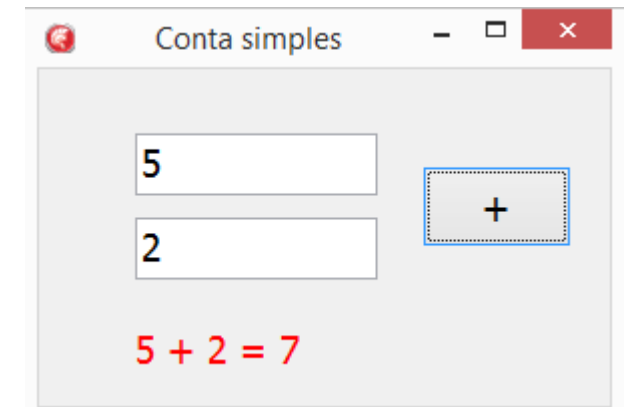
- Assim a variável total recebe o valor da soma das várias n1 e n2.
- Agora precisamos exibir o resultado na label correspondente...

```
procedure TfrmConta.btnCalculoClick(Sender: TObject);  
var  
    n1, n2, total : Integer;  
begin  
    n1 := StrToInt(edtn1.Text);  
    n2 := StrToInt(edtn2.Text);  
     total := n1 + n2;  
end;
```

Cálculos com variáveis – Ex 5

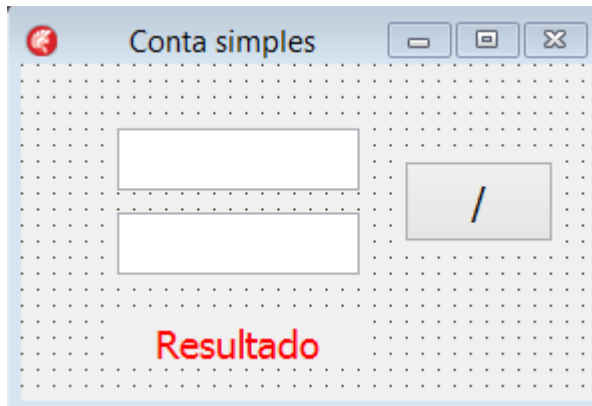
- Para armazenar o resultado na label é necessário converter os valores para o tipo String, neste exemplo estamos convertendo valores do tipo inteiro para String. (IntToStr).

```
procedure TfrmConta.btnCalculoClick(Sender: TObject);  
var  
    n1, n2, total : Integer;  
begin  
    n1 := StrToInt(edtn1.Text);  
    n2 := StrToInt(edtn2.Text);  
  
    total := n1 + n2;  
  
    lblTotal.Caption := IntToStr(total);  
  
end;
```



Usando variáveis do tipo Real - Ex 6

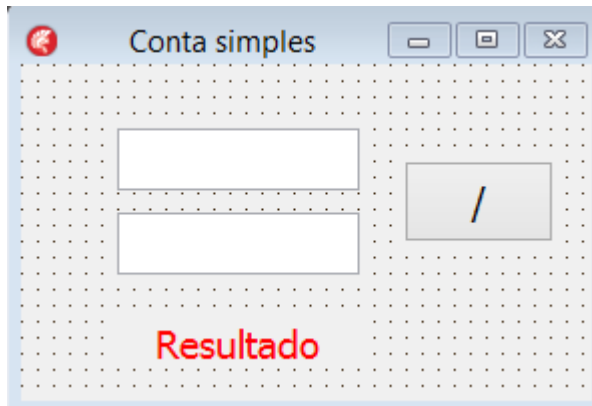
- Para realizar uma conta de divisão será necessário mudar o tipo da variável para real, pois o resultado da divisão pode ter casas decimais.



```
procedure TfrmConta.btnCalculoClick(Sender: TObject);  
var  
    n1, n2, total : Real;   
begin  
    n1 := StrToFloat(edtn1.Text);  
    n2 := StrToFloat(edtn2.Text);  
  
    total := n1 / n2;  
  
    lblTotal.Caption := FloatToStr(n1) + ' ÷ ' + FloatToStr(n2) + ' = ' +  
                        FormatFloat('0.00',total);  
  
end;
```

Usando variáveis do tipo Real - Ex 6

- Os valores digitados nos edits precisam ser convertidos para o tipo da variável usada, neste caso iremos usar (StrToFloat).



```
procedure TfrmConta.btnCalculoClick(Sender: TObject);
var
  n1, n2, total : Real;
begin
  n1 := StrToFloat(edtn1.Text);
  n2 := StrToFloat(edtn2.Text);

  total := n1 / n2;

  lblTotal.Caption := FloatToStr(n1) + ' / ' + FloatToStr(n2) + ' = ' +
    FormatFloat('0.00',total);
end;
```

Usando variáveis do tipo Real - Ex 6

- Após armazenarmos os valores nas respectivas variáveis iremos realizar o cálculo.



```
procedure TfrmConta.btnCalculoClick(Sender: TObject);
var
  n1, n2, total : Real;
begin
  n1 := StrToFloat(edtn1.Text);
  n2 := StrToFloat(edtn2.Text);

  total := n1 / n2;

  lblTotal.Caption := FloatToStr(n1) + ' + ' + FloatToStr(n2) + ' = ' +
    FormatFloat('0.00',total);
end;
```

Usando variáveis do tipo Real - Ex 6

- Em seguida iremos enviar o resultado para o label que irá exibi-lo.
- Existe outra forma de convertermos valores, o comando FormatFloat, ele permite que sejam definidos o número de casas decimais que queremos exibir. Caso não seja necessário controlar o número de casas decimais podemos utilizar o comando (FloatToStr).



```
procedure TfrmConta.btnCalculoClick(Sender: TObject);
var
  n1, n2, total : Real;
begin
  n1 := StrToFloat(edtn1.Text);
  n2 := StrToFloat(edtn2.Text);

  total := n1 / n2;

  lblTotal.Caption := FloatToStr(n1) + ' + ' + FloatToStr(n2) + ' = ' +
    FormatFloat('0.00',total);

end;
```



Div e Mod - Ex 7

- Agora crie um programa para identificar a diferença entre os operadores Div e Mod.



```
procedure TfrmConta.btnDivClick(Sender: TObject);
var
  n1, n2, total : Integer;
begin
  n1 := StrToInt(edtn1.Text);
  n2 := StrToInt(edtn2.Text);

  total := n1 div n2;

  lblTotal.Caption := IntToStr(n1) + ' Div ' + IntToStr(n2) + ' = ' +
    IntToStr(total);

end;

procedure TfrmConta.btnModClick(Sender: TObject);
var
  n1, n2, total : Integer;
begin
  n1 := StrToInt(edtn1.Text);
  n2 := StrToInt(edtn2.Text);

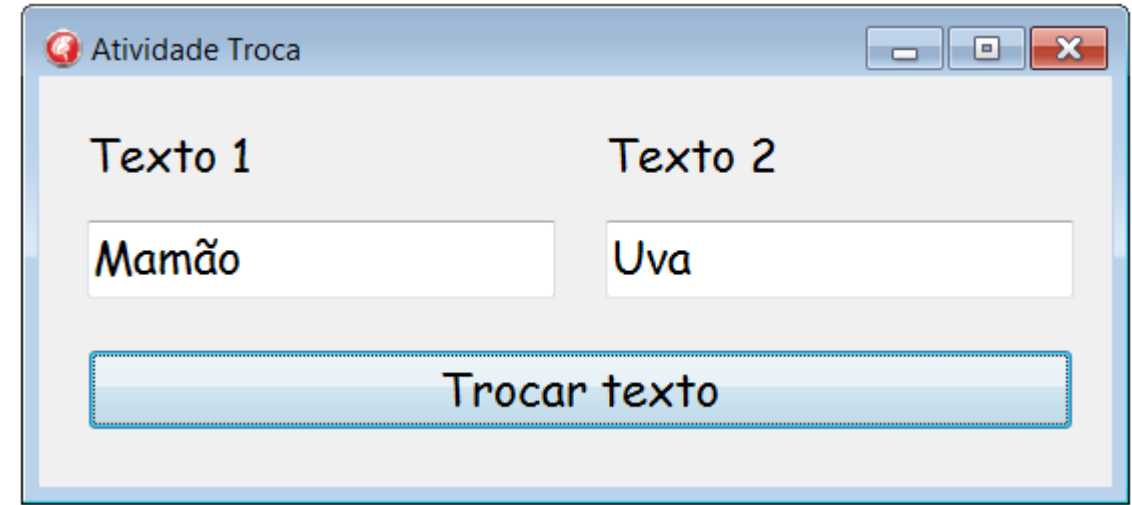
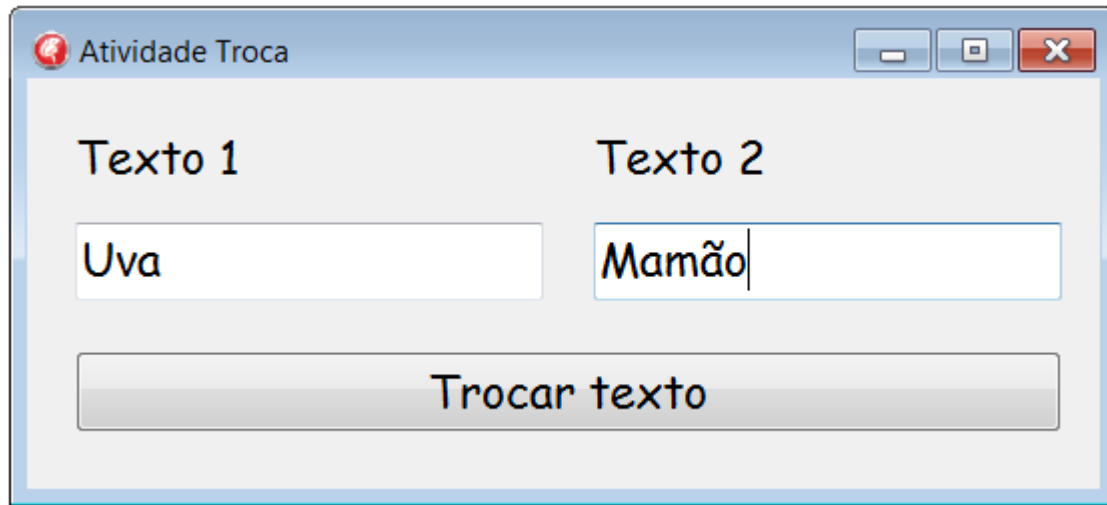
  total := n1 mod n2;

  lblTotal.Caption := IntToStr(n1) + ' Mod ' + IntToStr(n2) + ' = ' +
    IntToStr(total);

end;
```

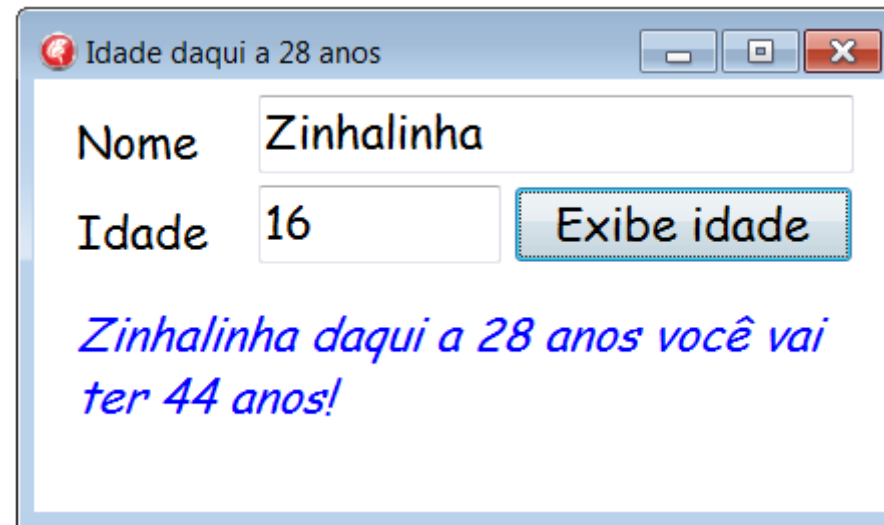
Exercícios

25. Crie um programa que receba dois valores e que ao clicar no botão trocar o conteúdo do mesmo seja trocado (o que faz parte de um Edit vai para o outro).



Exercícios

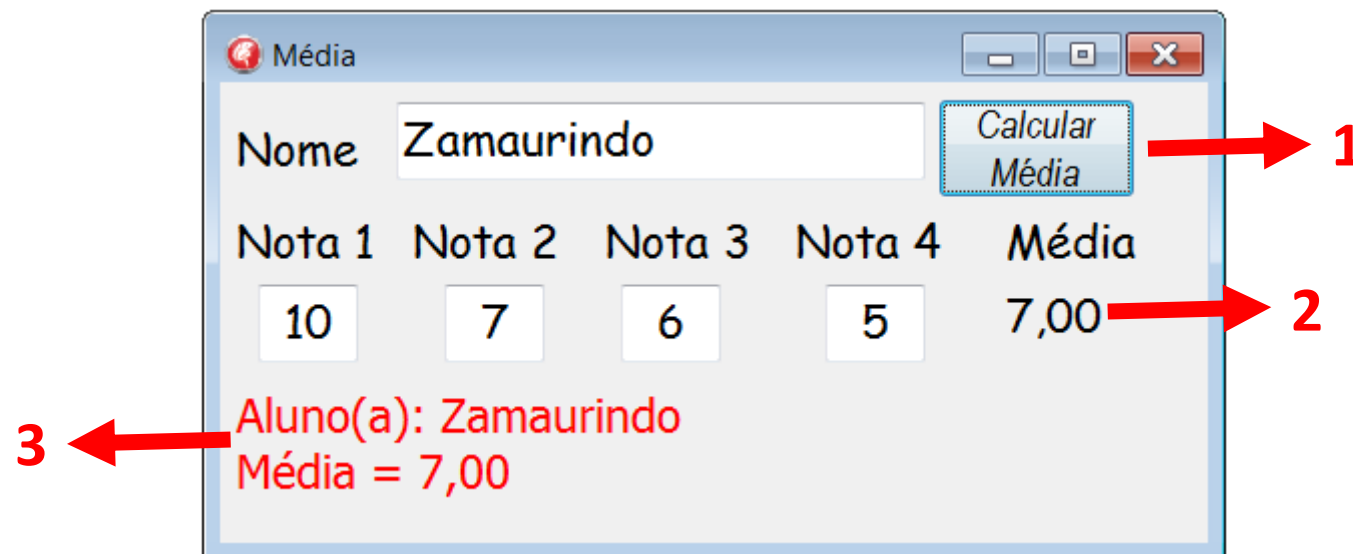
26. Crie um programa que recena o nome e idade de uma pessoa, e exiba uma mensagem contendo o nome e a idade da mesma pessoa daqui a 28 anos.



The image shows a screenshot of a Java Swing window titled "Idade daqui a 28 anos". The window has a standard title bar with minimize, maximize, and close buttons. Inside the window, there is a form with two input fields: "Nome" (containing "Zinhalinha") and "Idade" (containing "16"). To the right of the "Idade" field is a button labeled "Exibe idade". Below the input fields, there is a message displayed in blue italicized text: "Zinhalinha daqui a 28 anos você vai ter 44 anos!".

Exercícios

27. Crie um programa que receba quatro notas, e ao clicar no botão 1- Calcular Média o calculo seja realizado. Em seguida ao dar um duplo clique na 2- nota que aparece na média, o programa exiba 3- o nome do aluno e sua média.



Exercícios

28. Faça um Programa que pergunte quanto você ganha por hora, o número de horas trabalhadas em cada dia do mês. Calcule e mostre o total do seu salário no referido mês e a média de horas trabalhadas no mês.

The screenshot shows a Python GUI application window titled "Calcular salário". It features a text input field for "\$ hora" with the value "10", a "Calcular" button, and two output labels: "Salário: R\$ 1000,00" and "Média de horas Mês: 3,33". Below these, there is a grid of 30 input fields for daily hours, numbered 1 to 30. The values entered are: 0, 4, 4, 4, 4, 4, 0, 0, 4, 4 for days 1-10; 4, 4, 4, 0, 0, 4, 4, 4, 4, 4 for days 11-20; and 0, 5, 6, 7, 10, 4, 4, 0, 0, 4 for days 21-30. A watermark "ewertonjsilva@gmail.com" is visible at the bottom.

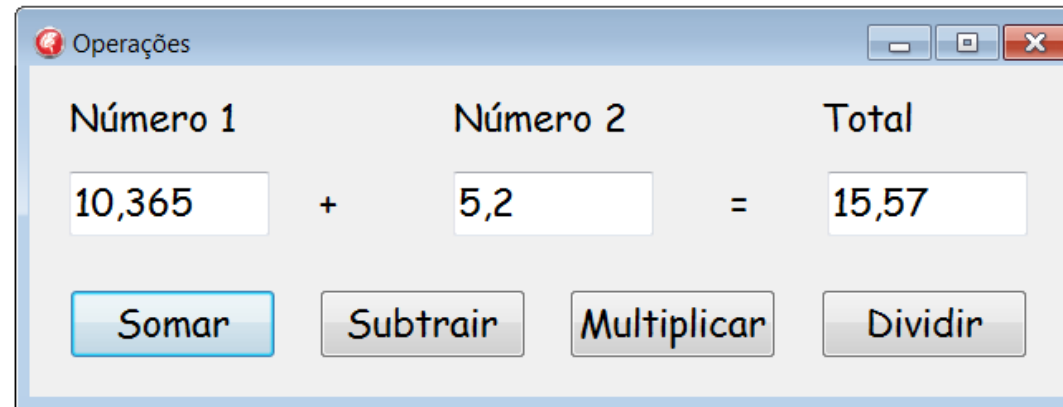
1	2	3	4	5	6	7	8	9	10
0	4	4	4	4	4	0	0	4	4

11	12	13	14	15	16	17	18	19	20
4	4	4	0	0	4	4	4	4	4

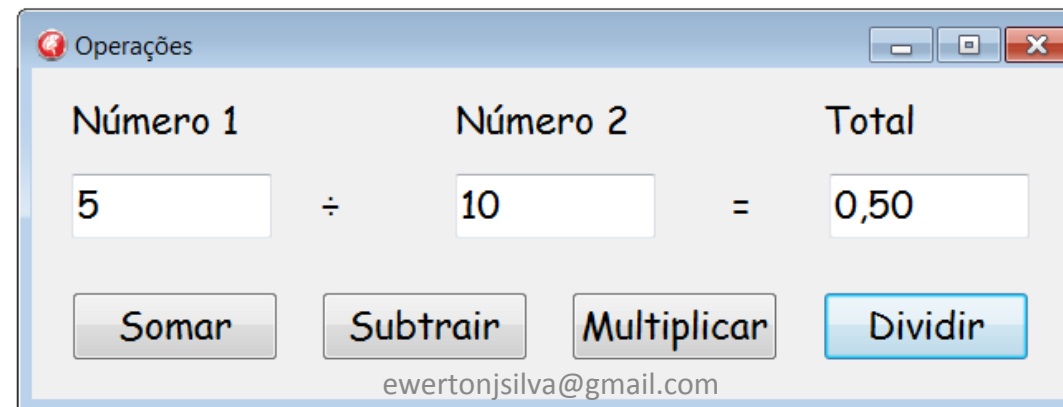
21	22	23	24	25	26	27	28	29	30
0	5	6	7	10	4	4	0	0	4

Exercícios

29. Crie um programa que realize as operações como do exemplo abaixo:



The screenshot shows a Windows application window titled "Operações". It contains three input fields: "Número 1" with the value "10,365", "Número 2" with the value "5,2", and "Total" with the value "15,57". Between the input fields are the operators "+", "=", and a space. Below the input fields are four buttons: "Somar" (highlighted in blue), "Subtrair", "Multiplicar", and "Dividir".



The screenshot shows a Windows application window titled "Operações". It contains three input fields: "Número 1" with the value "5", "Número 2" with the value "10", and "Total" with the value "0,50". Between the input fields are the operators "÷", "=", and a space. Below the input fields are four buttons: "Somar", "Subtrair", "Multiplicar", and "Dividir" (highlighted in blue). At the bottom of the window, the email address "ewertonjsilva@gmail.com" is displayed.

Exercícios

30. Faça um Programa que pergunte quanto você ganha por hora e o número de horas trabalhadas no mês. Calcule e mostre o total do seu salário no referido mês, sabendo-se que são descontados 11% para o Imposto de Renda, 8% para o INSS e 5% para o sindicato, faça um programa que nos dê:

- salário bruto.
- Quanto pagou de IR.
- quanto pagou ao INSS.
- quanto pagou ao sindicato.
- o salário líquido.

+	Salário Bruto
-	IR (11%)
-	INSS (8%)
-	Sindicato (5%)
=	Salário Líquido

Obs.: Salário Bruto - Descontos = Salário Líquido.

Exercícios

30. Exemplo

Dados	
Nome	Porunga
Valor hora	20
Horas no mês	160

Pagamento	
Salário Bruto:	+ 3200,00
Imposto de Renda:	- 352,00
INSS:	- 256,00
Sindicato:	- 160,00
Salário Líquido:	= 2432,00

Calcular Salário **Limpar**

Bibliografia

- Ana Paula Muniz; Diogo Rocha Ferreira de Menezes. Artigo Evolução da linguagem Delphi – Web Publicação em 13/08/ 2014. Disponível em <http://pt.slideshare.net/diogorochamenezes/evolucao-da-linguagem-delphi-artigo>. Acesso em 26/04/2015
- ANSELMO, FERNANDO ANTONIO F. Desvendando o Caminho das Pedras. Web Publicação de 1995-97. Disponível em <http://www.greantoniobraga.seed.pr.gov.br/redeescola/escolas/13/870/10/arquivos/File/Adenildo/Biblia-Delphi-7-PtBr.pdf>. Acesso em 27/04/2015
- BARCELOS, ÉRICA. Linguagem de Programação Delphi Orientada a objeto – Web Publicação de Fevereiro de 2012. Disponível em <http://www.simonson.br/its/pdf/apostilas/base-tecnica/2/linguagem-de-programacao-1-capitulo-2-ano-informatica.pdf> e <http://www.simonson.br/its/pdf/apostilas/base-tecnica/2/linguagem-de-programacao-2-capitulo-2-ano-informatica.pdf>. Acesso em 12/03/2014.
- LEÃO, MARCELO. Borland Delphi 7 curso completo. Editora Axcel Books, 1ª Edição, 2003.
- PEREIRA, PAULO ROBERTO ALVES. Desenvolvendo aplicações orientadas a objetos com Borland Delphi. Web Publicação de 2002. Disponível em <http://www2.fateb.br/ftp/apostilas/Delphi/OO-Delphi.pdf>. Acesso em 19/03/2014.
- YAMAZACK, WESLEY. Recursos de Compilação no Delphi. Web Publicação. <http://www.devmedia.com.br/recursos-de-compilacao-no-delphi/16964>. Acesso em 25/01/2016